

MongoDB Case Study: SmartRetail Order Management System

1. Case-study overview

Organization

SmartRetail Pvt. Ltd. operates an online shopping platform that sells electronics, home appliances, books and fashion products.

The company currently stores customer, product and order information in different files. Because product attributes and order structures change frequently, maintaining the data has become difficult.

SmartRetail has decided to implement MongoDB to manage:

- Customer profiles
- Product catalogues
- Inventory
- Customer orders
- Payment information
- Delivery information
- Sales reports

The development team must create and manage the database through the **MongoDB Shell** — `mongosh`.

2. Learning objectives

After completing the case study, participants should be able to:

1. Connect to MongoDB using `mongosh`.
 2. Create and select a database.
 3. Create and inspect collections.
 4. Work with primary MongoDB BSON data types.
 5. Insert single and multiple documents.
 6. retrieve documents using filters and projections.
 7. Apply comparison, logical, array and element operators.
 8. Update individual and multiple documents.
 9. Delete documents safely.
 10. Build aggregation pipelines.
 11. Create and analyse indexes.
 12. Export, import, back up, restore and monitor MongoDB data.
-

3. Estimated duration

Module	Duration
MongoDB Shell and database setup	30 minutes
BSON data types and data insertion	45 minutes
CRUD operations and queries	75 minutes
Updating and deleting documents	45 minutes
Aggregation framework	60 minutes
Indexes and query analysis	30 minutes
MongoDB utilities	30 minutes
Challenge and assessment	30 minutes
Total	5 hours 15 minutes

Part 1: Starting MongoDB

4. Start MongoDB with Docker

Run the following command from the operating-system terminal:

```
docker run -d
  --name mongoddb-container
  -p 27017:27017
  -e MONGO_INITDB_ROOT_USERNAME=admin
  -e MONGO_INITDB_ROOT_PASSWORD=admin123
  -v mongoddb-data:/data/db
  mongo
```

Verify that the container is running:

```
docker ps
```

Open the MongoDB Shell:

```
docker exec -it mongodb-container  
mongosh -u admin -p admin123 --authenticationDatabase admin
```

Part 2: Introduction to the MongoDB Shell

5. Shell exploration

Run the following commands inside `mongosh`.

Check the current database:

```
db
```

Display available databases:

```
show dbs
```

Display shell help:

```
help
```

Display database-level help:

```
db.help()
```

Check the MongoDB server version:

```
db.version()
```

Clear the terminal:

```
cls
```

On macOS or Linux, the keyboard shortcut may also be used:

```
Ctrl + L
```

6. Create the case-study database

Select the SmartRetail database:

```
use smartRetailDB
```

Check the selected database:

```
db
```

Create the required collections:

```
db.createCollection("customers")  
db.createCollection("products")  
db.createCollection("orders")
```

View the collections:

```
show collections
```

Alternative command:

```
db.getCollectionNames()
```

Inspect collection-level help:

```
db.products.help()
```

Part 3: Primary MongoDB Data Types

MongoDB stores data as BSON documents. The following types will be used in the case study.

Data type	Business example
String	Customer name or product name
Integer	Age, stock or quantity
Decimal	Product price or total order value
Boolean	Premium membership
Date	Registration or order date
Array	Interests, tags or order items
Embedded document	Address or payment details
ObjectId	MongoDB-generated identifier
Null	Information not currently available

7. BSON data-type demonstration

Insert a demonstration customer:

```
db.customers.insertOne({
  customerId: "C100",
  name: "Test Customer",
  age: NumberInt(30),
  accountBalance: NumberDecimal("2500.50"),
  premiumMember: true,
  registeredOn: ISODate("2026-01-10T10:30:00Z"),
  interests: ["Electronics", "Books"],
  address: {
    city: "Pune",
    state: "Maharashtra",
    pinCode: "411057"
  },
  referredBy: null
})
```

Retrieve the document:

```
db.customers.findOne({
  customerId: "C100"
})
```

Check selected field types:

```

db.customers.aggregate([
  {
    $match: {
      customerId: "C100"
    }
  },
  {
    $project: {
      customerId: 1,
      nameType: {
        $type: "$name"
      },
      ageType: {
        $type: "$age"
      },
      balanceType: {
        $type: "$accountBalance"
      },
      registrationType: {
        $type: "$registeredOn"
      }
    }
  }
])

```

Delete the temporary demonstration document:

```

db.customers.deleteOne({
  customerId: "C100"
})

```

Part 4: Creating the Business Dataset

8. Insert customer documents

```

db.customers.insertMany([
  {
    customerId: "C101",
    name: "Aarav Sharma",
    email: "aarav@example.com",
    age: NumberInt(29),
    premiumMember: true,
  }
])

```

```

    registeredOn: ISODate("2025-10-15T09:30:00Z"),
    interests: ["Electronics", "Books"],
    address: {
      city: "Pune",
      state: "Maharashtra",
      pinCode: "411057"
    },
    lastLogin: ISODate("2026-07-10T08:15:00Z")
  },
  {
    customerId: "C102",
    name: "Meera Joshi",
    email: "meera@example.com",
    age: NumberInt(34),
    premiumMember: false,
    registeredOn: ISODate("2026-01-20T11:00:00Z"),
    interests: ["Fashion", "Home"],
    address: {
      city: "Mumbai",
      state: "Maharashtra",
      pinCode: "400076"
    },
    lastLogin: ISODate("2026-07-08T17:20:00Z")
  },
  {
    customerId: "C103",
    name: "Rohan Verma",
    email: "rohan@example.com",
    age: NumberInt(41),
    premiumMember: true,
    registeredOn: ISODate("2025-06-11T12:45:00Z"),
    interests: ["Electronics", "Home"],
    address: {
      city: "Bengaluru",
      state: "Karnataka",
      pinCode: "560001"
    },
    lastLogin: null
  },
  {
    customerId: "C104",
    name: "Priya Nair",
    email: "priya@example.com",
    age: NumberInt(25),
    premiumMember: false,
    registeredOn: ISODate("2026-03-05T10:10:00Z"),
    interests: ["Books", "Fashion"],
    address: {

```

```

        city: "Pune",
        state: "Maharashtra",
        pinCode: "411045"
    },
    lastLogin: ISODate("2026-07-11T15:40:00Z")
},
{
    customerId: "C105",
    name: "Kabir Singh",
    email: "kabir@example.com",
    age: NumberInt(37),
    premiumMember: true,
    registeredOn: ISODate("2025-12-22T14:30:00Z"),
    interests: ["Electronics", "Fitness"],
    address: {
        city: "Delhi",
        state: "Delhi",
        pinCode: "110001"
    },
    lastLogin: ISODate("2026-07-09T19:10:00Z")
}
])

```

Verify the insertion:

```
db.customers.countDocuments()
```

9. Insert product documents

```

db.products.insertMany([
  {
    productId: "P1001",
    name: "Business Laptop",
    category: "Electronics",
    brand: "TechPro",
    price: NumberDecimal("65000.00"),
    stock: NumberInt(25),
    active: true,
    tags: ["Laptop", "Business", "Portable"],
    rating: 4.5,
    specifications: {
      processor: "Intel Core i7",
      ram: "16 GB",

```



```

        storage: "512 GB SSD"
    },
    {
        productId: "P1002",
        name: "Wireless Mouse",
        category: "Electronics",
        brand: "ClickTech",
        price: NumberDecimal("1200.00"),
        stock: NumberInt(80),
        active: true,
        tags: ["Mouse", "Wireless", "Accessory"],
        rating: 4.2,
        specifications: {
            connectivity: "Bluetooth",
            batteryLife: "12 months"
        }
    },
    {
        productId: "P1003",
        name: "MongoDB Fundamentals",
        category: "Books",
        brand: "Tech Publications",
        price: NumberDecimal("750.00"),
        stock: NumberInt(50),
        active: true,
        tags: ["Book", "Database", "MongoDB"],
        rating: 4.7,
        specifications: {
            author: "R. Kumar",
            pages: NumberInt(420)
        }
    },
    {
        productId: "P1004",
        name: "Smart Fitness Watch",
        category: "Fitness",
        brand: "FitTrack",
        price: NumberDecimal("4500.00"),
        stock: NumberInt(35),
        active: true,
        tags: ["Watch", "Fitness", "Bluetooth"],
        rating: 4.1,
        specifications: {
            waterResistant: true,
            batteryLife: "7 days"
        }
    }
},

```

```

{
  productId: "P1005",
  name: "Cotton Jacket",
  category: "Fashion",
  brand: "UrbanStyle",
  price: NumberDecimal("2800.00"),
  stock: NumberInt(40),
  active: true,
  tags: ["Jacket", "Winter", "Cotton"],
  rating: 3.9,
  specifications: {
    availableSizes: ["M", "L", "XL"],
    color: "Black"
  }
},
{
  productId: "P1006",
  name: "Electric Kettle",
  category: "Home",
  brand: "HomeEase",
  price: NumberDecimal("1800.00"),
  stock: NumberInt(0),
  active: false,
  tags: ["Kitchen", "Kettle", "Appliance"],
  rating: 4.3,
  specifications: {
    capacity: "1.5 litres",
    power: "1500 watts"
  }
}
]
)

```

10. Insert order documents

```

db.orders.insertMany([
  {
    orderId: "O5001",
    customerId: "C101",
    orderedOn: ISODate("2026-07-01T10:15:00Z"),
    status: "DELIVERED",
    items: [
      {
        productId: "P1001",
        productName: "Business Laptop",

```

```

        category: "Electronics",
        quantity: NumberInt(1),
        unitPrice: NumberDecimal("65000.00")
    },
    {
        productId: "P1002",
        productName: "Wireless Mouse",
        category: "Electronics",
        quantity: NumberInt(1),
        unitPrice: NumberDecimal("1200.00")
    }
],
totalAmount: NumberDecimal("66200.00"),
payment: {
    mode: "CARD",
    status: "PAID",
    transactionId: "TXN10001"
},
delivery: {
    city: "Pune",
    deliveredOn: ISODate("2026-07-04T13:00:00Z")
}
},
{
    orderId: "O5002",
    customerId: "C102",
    orderedOn: ISODate("2026-07-03T14:20:00Z"),
    status: "SHIPPED",
    items: [
        {
            productId: "P1005",
            productName: "Cotton Jacket",
            category: "Fashion",
            quantity: NumberInt(2),
            unitPrice: NumberDecimal("2800.00")
        }
    ],
    totalAmount: NumberDecimal("5600.00"),
    payment: {
        mode: "UPI",
        status: "PAID",
        transactionId: "TXN10002"
    },
    delivery: {
        city: "Mumbai",
        deliveredOn: null
    }
}
},

```

```

{
  orderId: "05003",
  customerId: "C103",
  orderedOn: ISODate("2026-07-05T09:35:00Z"),
  status: "PROCESSING",
  items: [
    {
      productId: "P1004",
      productName: "Smart Fitness Watch",
      category: "Fitness",
      quantity: NumberInt(1),
      unitPrice: NumberDecimal("4500.00")
    },
    {
      productId: "P1003",
      productName: "MongoDB Fundamentals",
      category: "Books",
      quantity: NumberInt(1),
      unitPrice: NumberDecimal("750.00")
    }
  ],
  totalAmount: NumberDecimal("5250.00"),
  payment: {
    mode: "CARD",
    status: "PAID",
    transactionId: "TXN10003"
  },
  delivery: {
    city: "Bengaluru",
    deliveredOn: null
  }
},
{
  orderId: "05004",
  customerId: "C104",
  orderedOn: ISODate("2026-07-06T17:45:00Z"),
  status: "CANCELLED",
  items: [
    {
      productId: "P1006",
      productName: "Electric Kettle",
      category: "Home",
      quantity: NumberInt(1),
      unitPrice: NumberDecimal("1800.00")
    }
  ],
  totalAmount: NumberDecimal("1800.00"),
  payment: {

```

```

        mode: "COD",
        status: "CANCELLED",
        transactionId: null
    },
    delivery: {
        city: "Pune",
        deliveredOn: null
    }
},
{
    orderId: "05005",
    customerId: "C105",
    orderedOn: ISODate("2026-07-08T12:10:00Z"),
    status: "DELIVERED",
    items: [
        {
            productId: "P1004",
            productName: "Smart Fitness Watch",
            category: "Fitness",
            quantity: NumberInt(2),
            unitPrice: NumberDecimal("4500.00")
        }
    ],
    totalAmount: NumberDecimal("9000.00"),
    payment: {
        mode: "UPI",
        status: "PAID",
        transactionId: "TXN10005"
    },
    delivery: {
        city: "Delhi",
        deliveredOn: ISODate("2026-07-11T11:30:00Z")
    }
},
{
    orderId: "05006",
    customerId: "C101",
    orderedOn: ISODate("2026-07-10T16:50:00Z"),
    status: "PROCESSING",
    items: [
        {
            productId: "P1003",
            productName: "MongoDB Fundamentals",
            category: "Books",
            quantity: NumberInt(2),
            unitPrice: NumberDecimal("750.00")
        }
    ]
},

```

```
totalAmount: NumberDecimal("1500.00"),
payment: {
  mode: "UPI",
  status: "PAID",
  transactionId: "TXN10006"
},
delivery: {
  city: "Pune",
  deliveredOn: null
}
}
1)
```

Verify all collection counts:

```
db.customers.countDocuments()
db.products.countDocuments()
db.orders.countDocuments()
```

Part 5: Basic Create and Read Operations

11. Insert one new customer

```
db.customers.insertOne({
  customerId: "C106",
  name: "Neha Patil",
  email: "neha@example.com",
  age: NumberInt(31),
  premiumMember: false,
  registeredOn: new Date(),
  interests: ["Books", "Home"],
  address: {
    city: "Nashik",
    state: "Maharashtra",
    pinCode: "422001"
  },
  lastLogin: null
})
```

12. Retrieve all products

```
db.products.find()
```

13. Retrieve one product

```
db.products.findOne({  
  productId: "P1001"  
})
```

14. Use projection

Display only product name, category and price:

```
db.products.find(  
  {},  
  {  
    _id: 0,  
    name: 1,  
    category: 1,  
    price: 1  
  }  
)
```

Exclude selected fields:

```
db.customers.find(  
  {},  
  {  
    _id: 0,  
    lastLogin: 0  
  }  
)
```

15. Sorting and limiting results

Display products from highest to lowest price:

```
db.products.find(
  {},
  {
    _id: 0,
    name: 1,
    price: 1
  }
).sort({
  price: -1
})
```

Display the three most expensive products:

```
db.products.find(
  {},
  {
    _id: 0,
    name: 1,
    price: 1
  }
).sort({
  price: -1
}).limit(3)
```

Display customers in alphabetical order:

```
db.customers.find(
  {},
  {
    _id: 0,
    customerId: 1,
    name: 1
  }
).sort({
  name: 1
})
```

16. Count and distinct operations

Count active products:


```
db.products.countDocuments({
  active: true
})
```

Display distinct product categories:

```
db.products.distinct("category")
```

Display the cities in which customers are registered:

```
db.customers.distinct("address.city")
```

Part 6: MongoDB Queries

17. Equality queries

Find all electronics products:

```
db.products.find({
  category: "Electronics"
})
```

Find all delivered orders:

```
db.orders.find({
  status: "DELIVERED"
})
```

18. Comparison operators

Greater than

Find products costing more than ₹3,000:

```
db.products.find({
  price: {
```

```
    $gt: NumberDecimal("3000.00")
  }
})
```

Greater than or equal to

Find products with at least 40 units in stock:

```
db.products.find({
  stock: {
    $gte: 40
  }
})
```

Less than

Find customers younger than 30:

```
db.customers.find({
  age: {
    $lt: 30
  }
})
```

Range query

Find products priced between ₹1,000 and ₹5,000:

```
db.products.find({
  price: {
    $gte: NumberDecimal("1000.00"),
    $lte: NumberDecimal("5000.00")
  }
})
```

Not equal

Find orders that have not been delivered:

```
db.orders.find({
  status: {
    $ne: "DELIVERED"
  }
})
```

```
}  
})
```

19. \$in and \$nin

Find products from the Electronics or Fitness category:

```
db.products.find(  
  category: {  
    $in: ["Electronics", "Fitness"]  
  }  
)
```

Exclude cancelled and delivered orders:

```
db.orders.find(  
  status: {  
    $nin: ["CANCELLED", "DELIVERED"]  
  }  
)
```

20. Logical operators

\$and

Find premium customers older than 30:

```
db.customers.find(  
  $and: [  
    {  
      premiumMember: true  
    },  
    {  
      age: {  
        $gt: 30  
      }  
    }  
  ]  
)
```

The same query can also be written as:

```
db.customers.find({
  premiumMember: true,
  age: {
    $gt: 30
  }
})
```

\$or

Find products that are either out of stock or inactive:

```
db.products.find({
  $or: [
    {
      stock: 0
    },
    {
      active: false
    }
  ]
})
```

Combined conditions

Find active Electronics or Fitness products costing less than ₹10,000:

```
db.products.find({
  active: true,
  category: {
    $in: ["Electronics", "Fitness"]
  },
  price: {
    $lt: NumberDecimal("10000.00")
  }
})
```

21. Query embedded documents

Find customers from Pune:

```
db.customers.find({
  "address.city": "Pune"
})
```

Find orders paid through UPI:

```
db.orders.find({
  "payment.mode": "UPI"
})
```

Find orders whose payment is complete:

```
db.orders.find({
  "payment.status": "PAID"
})
```

22. Query arrays

Find customers interested in Electronics:

```
db.customers.find({
  interests: "Electronics"
})
```

Find products tagged as Bluetooth products:

```
db.products.find({
  tags: "Bluetooth"
})
```

Find customers interested in both Electronics and Books:

```
db.customers.find({
  interests: {
    $all: ["Electronics", "Books"]
  }
})
```

Find products having exactly three tags:

```
db.products.find({
  tags: {
    $size: 3
  }
})
```

23. Query arrays of embedded documents

Find orders containing product `P1004`:

```
db.orders.find({
  "items.productId": "P1004"
})
```

Find orders containing an item with a quantity of at least two:

```
db.orders.find({
  items: {
    $elemMatch: {
      quantity: {
        $gte: 2
      }
    }
  }
})
```

Find an order containing two or more Fitness items:

```
db.orders.find({
  items: {
    $elemMatch: {
      category: "Fitness",
      quantity: {
        $gte: 2
      }
    }
  }
})
```

24. Regular-expression query

Find customers whose names start with the letter `P`:

```
db.customers.find({
  name: {
    $regex: "^P",
    $options: "i"
  }
})
```

Find products containing the word `smart`:

```
db.products.find({
  name: {
    $regex: "smart",
    $options: "i"
  }
})
```

25. Existence and type queries

Find orders for which a transaction ID exists and is not null:

```
db.orders.find({
  "payment.transactionId": {
    $exists: true,
    $ne: null
  }
})
```

Find customers whose last login is null:

```
db.customers.find({
  lastLogin: null
})
```

Find documents where age is stored as an integer:

```
db.customers.find({
  age: {
    $type: "int"
  }
})
```

Part 7: Updating Documents

MongoDB provides `updateOne()` for one matching document and `updateMany()` for multiple matching documents.

26. Update one field using `$set`

Make customer `C102` a premium member:

```
db.customers.updateOne(
  {
    customerId: "C102"
  },
  {
    $set: {
      premiumMember: true
    }
  }
)
```

Verify the update:

```
db.customers.findOne({
  customerId: "C102"
})
```

27. Update multiple fields

Update an order after shipping:

```
db.orders.updateOne(
  {
```



```
    orderId: "05003"
  },
  {
    $set: {
      status: "SHIPPED",
      "delivery.shippedOn": new Date(),
      "delivery.courier": "SmartExpress"
    }
  }
}
```

28. Increment a numeric value using `$inc`

Add ten units to the Wireless Mouse stock:

```
db.products.updateOne(
  {
    productId: "P1002"
  },
  {
    $inc: {
      stock: 10
    }
  }
)
```

Decrease stock after selling two fitness watches:

```
db.products.updateOne(
  {
    productId: "P1004",
    stock: {
      $gte: 2
    }
  },
  {
    $inc: {
      stock: -2
    }
  }
)
```

The stock condition prevents the operation when fewer than two units are available.

29. Add an element to an array using `$push`

```
db.customers.updateOne(
  {
    customerId: "C104"
  },
  {
    $push: {
      interests: "MongoDB"
    }
  }
)
```

30. Prevent duplicate array values using `$addToSet`

```
db.customers.updateOne(
  {
    customerId: "C104"
  },
  {
    $addToSet: {
      interests: "MongoDB"
    }
  }
)
```

Running this command repeatedly will not insert duplicate `"MongoDB"` entries.

31. Remove an array element using `$pull`

```
db.customers.updateOne(
  {
    customerId: "C104"
  },
  {
    $pull: {
      interests: "MongoDB"
    }
  }
)
```

```
}  
)
```

32. Remove a field using `$unset`

Remove the courier field from order `05003`:

```
db.orders.updateOne(  
  {  
    orderId: "05003"  
  },  
  {  
    $unset: {  
      "delivery.courier": ""  
    }  
  }  
)
```

33. Update multiple documents

Provide a ten-percent discount flag to all active Fashion products:

```
db.products.updateMany(  
  {  
    category: "Fashion",  
    active: true  
  },  
  {  
    $set: {  
      discountPercentage: NumberInt(10)  
    }  
  }  
)
```

Add a loyalty level to premium customers:

```
db.customers.updateMany(  
  {  
    premiumMember: true  
  },
```

```
{
  $set: {
    loyaltyLevel: "GOLD"
  }
}
```

34. Rename a field

Rename `rating` to `averageRating`:

```
db.products.updateMany(
  {},
  {
    $rename: {
      rating: "averageRating"
    }
  }
)
```

35. Replace an entire document

Create a temporary product:

```
db.products.insertOne({
  productId: "P9999",
  name: "Temporary Product",
  price: NumberDecimal("100.00")
})
```

Replace it:

```
db.products.replaceOne(
  {
    productId: "P9999"
  },
  {
    productId: "P9999",
    name: "USB-C Cable",
    category: "Electronics",
  }
)
```

```
    brand: "ConnectPlus",
    price: NumberDecimal("499.00"),
    stock: NumberInt(100),
    active: true,
    tags: ["Cable", "USB-C", "Accessory"]
  }
}
```

`replaceOne()` replaces the complete document except for its existing `_id`.

36. Upsert operation

Update a product, or insert it when it does not exist:

```
db.products.updateOne(
  {
    productId: "P1007"
  },
  {
    $set: {
      name: "Laptop Stand",
      category: "Electronics",
      brand: "DeskPro",
      price: NumberDecimal("1500.00"),
      stock: NumberInt(20),
      active: true
    }
  },
  {
    upsert: true
  }
)
```

Part 8: Delete Operations

37. Safe deletion process

Before deleting documents, always inspect the matching records.

Preview inactive products:

```
db.products.find({
  active: false
})
```

Count matching products:

```
db.products.countDocuments({
  active: false
})
```

Only after verification should the delete command be executed.

38. Delete one document

Delete the temporary product:

```
db.products.deleteOne({
  productId: "P9999"
})
```

39. Delete multiple documents

Delete cancelled orders:

```
db.orders.deleteMany({
  status: "CANCELLED"
})
```

Verify the result:

```
db.orders.find({
  status: "CANCELLED"
})
```

40. Delete documents using a date condition

The following command demonstrates how old cancelled orders could be removed:

```
db.orders.deleteMany({
  status: "CANCELLED",
  orderedOn: {
    $lt: ISODate("2025-01-01T00:00:00Z")
  }
})
```

41. Empty a collection without deleting it

Do not execute this command unless the data can be removed:

```
db.orders.deleteMany({})
```

This removes all documents but keeps the collection and its indexes.

42. Drop a collection

Do not execute during the main lab:

```
db.orders.drop()
```

This removes the complete collection, its documents and its indexes.

Part 9: Aggregation Framework

Aggregation pipelines process documents through ordered stages and can filter, group, calculate, project and sort results.

43. Count orders by status

```
db.orders.aggregate([
  {
    $group: {
```

```
    _id: "$status",
    numberOfOrders: {
      $sum: 1
    }
  },
  {
    $sort: {
      numberOfOrders: -1
    }
  }
]
1)
```

44. Calculate total sales by order status

```
db.orders.aggregate([
  {
    $group: {
      _id: "$status",
      orderCount: {
        $sum: 1
      },
      totalOrderValue: {
        $sum: "$totalAmount"
      },
      averageOrderValue: {
        $avg: "$totalAmount"
      }
    }
  },
  {
    $sort: {
      totalOrderValue: -1
    }
  }
]
1)
```

45. Calculate delivered-order revenue

```
db.orders.aggregate([
  {
```



```

    $match: {
      status: "DELIVERED"
    }
  },
  {
    $group: {
      _id: null,
      deliveredOrders: {
        $sum: 1
      },
      deliveredRevenue: {
        $sum: "$totalAmount"
      }
    }
  },
  {
    $project: {
      _id: 0,
      deliveredOrders: 1,
      deliveredRevenue: 1
    }
  }
}
])

```

46. Calculate city-wise order value

```

db.orders.aggregate([
  {
    $group: {
      _id: "$delivery.city",
      numberOfOrders: {
        $sum: 1
      },
      totalOrderValue: {
        $sum: "$totalAmount"
      }
    }
  },
  {
    $sort: {
      totalOrderValue: -1
    }
  }
])

```

47. Analyse individual order items using `$unwind`

```
db.orders.aggregate([
  {
    $unwind: "$items"
  },
  {
    $project: {
      _id: 0,
      orderId: 1,
      customerId: 1,
      productName: "$items.productName",
      category: "$items.category",
      quantity: "$items.quantity",
      unitPrice: "$items.unitPrice"
    }
  }
])
```

48. Calculate category-wise sales

```
db.orders.aggregate([
  {
    $match: {
      status: {
        $ne: "CANCELLED"
      }
    }
  },
  {
    $unwind: "$items"
  },
  {
    $group: {
      _id: "$items.category",
      totalUnitsSold: {
        $sum: "$items.quantity"
      },
      totalRevenue: {
        $sum: {
          $multiply: [
            "$items.quantity",

```

```

        "$items.unitPrice"
    ]
  }
}
},
{
  $sort: {
    totalRevenue: -1
  }
}
]
)

```

49. Find the best-selling products

```

db.orders.aggregate([
  {
    $match: {
      status: {
        $ne: "CANCELLED"
      }
    }
  },
  {
    $unwind: "$items"
  },
  {
    $group: {
      _id: {
        productId: "$items.productId",
        productName: "$items.productName"
      },
      totalQuantitySold: {
        $sum: "$items.quantity"
      },
      totalRevenue: {
        $sum: {
          $multiply: [
            "$items.quantity",
            "$items.unitPrice"
          ]
        }
      }
    }
  }
]
)

```

```
    },
    {
      $sort: {
        totalQuantitySold: -1
      }
    },
    {
      $limit: 3
    }
  ]
})
```

50. Add calculated fields

Calculate the number of items contained in each order:

```
db.orders.aggregate([
  {
    $addFields: {
      numberOfDifferentProducts: {
        $size: "$items"
      },
      totalQuantity: {
        $sum: "$items.quantity"
      }
    }
  },
  {
    $project: {
      _id: 0,
      orderId: 1,
      status: 1,
      numberOfDifferentProducts: 1,
      totalQuantity: 1,
      totalAmount: 1
    }
  }
])
```

51. Categorise orders using `$switch`

```
db.orders.aggregate([
  {
    $project: {
      _id: 0,
      orderId: 1,
      totalAmount: 1,
      orderCategory: {
        $switch: {
          branches: [
            {
              case: {
                $gte: [
                  "$totalAmount",
                  NumberDecimal("50000.00")
                ]
              },
              then: "HIGH VALUE"
            },
            {
              case: {
                $gte: [
                  "$totalAmount",
                  NumberDecimal("5000.00")
                ]
              },
              then: "MEDIUM VALUE"
            }
          ],
          default: "LOW VALUE"
        }
      }
    }
  }
])
```

52. Join orders with customers using `$lookup`

```
db.orders.aggregate([
  {
    $lookup: {
      from: "customers",
```

```

        localField: "customerId",
        foreignField: "customerId",
        as: "customerDetails"
    }
},
{
    $unwind: "$customerDetails"
},
{
    $project: {
        _id: 0,
        orderId: 1,
        customerId: 1,
        customerName: "$customerDetails.name",
        customerCity: "$customerDetails.address.city",
        status: 1,
        totalAmount: 1
    }
},
{
    $sort: {
        totalAmount: -1
    }
}
]
])

```

53. Customer-wise order summary

```

db.orders.aggregate([
    {
        $match: {
            status: {
                $ne: "CANCELLED"
            }
        }
    },
    {
        $group: {
            _id: "$customerId",
            totalOrders: {
                $sum: 1
            },
            totalSpent: {
                $sum: "$totalAmount"
            }
        }
    }
])

```

```

    },
    averageOrderValue: {
      $avg: "$totalAmount"
    }
  }
},
{
  $lookup: {
    from: "customers",
    localField: "_id",
    foreignField: "customerId",
    as: "customer"
  }
},
{
  $unwind: "$customer"
},
{
  $project: {
    _id: 0,
    customerId: "$_id",
    customerName: "$customer.name",
    totalOrders: 1,
    totalSpent: 1,
    averageOrderValue: 1
  }
},
{
  $sort: {
    totalSpent: -1
  }
}
])

```

Part 10: Indexes

54. View existing indexes

```
db.customers.getIndexes()
```

Every collection normally contains an index for `_id`.

55. Create a unique index

Customer email addresses must not be duplicated:

```
db.customers.createIndex(  
  {  
    email: 1  
  },  
  {  
    unique: true  
  }  
)
```

Test the constraint by attempting to insert a customer with an existing email address.

The operation should fail with a duplicate-key error.

56. Create a single-field index

Create an index for product categories:

```
db.products.createIndex({  
  category: 1  
})
```

57. Create a compound index

The operations team frequently searches orders by status and then sorts them by order date.

```
db.orders.createIndex({  
  status: 1,  
  orderedOn: -1  
})
```

View indexes:

```
db.orders.getIndexes()
```

58. Examine query execution

Analyse the following query:

```
db.orders.explain("executionStats").find({
  status: "DELIVERED"
}).sort({
  orderedOn: -1
})
```

Participants should examine:

- winningPlan
- executionStages
- totalDocsExamined
- totalKeysExamined
- nReturned
- IXSCAN
- COLLSCAN

Interpretation

- IXSCAN indicates that an index was scanned.
- COLLSCAN indicates that the collection was scanned.
- A well-selected index generally reduces the number of documents examined.
- An index should support actual application query patterns.

59. Create a price-search index

```
db.products.createIndex({
  category: 1,
  price: 1
})
```

Analyse a matching query:

```
db.products.explain("executionStats").find({
  category: "Electronics",
  price: {
    $lt: NumberDecimal("10000.00")
  }
})
```

```
}  
})
```

60. Drop an index

List indexes first:

```
db.products.getIndexes()
```

Drop the category index:

```
db.products.dropIndex("category_1")
```

Do not drop the `_id` index.

Part 11: MongoDB Shell Utilities

61. Database information

Run inside `mongosh`:

```
db.stats()
```

Collection information:

```
db.products.stats()
```

Collection names:

```
db.getCollectionNames()
```

Current user connection information:

```
db.runCommand({
  connectionStatus: 1
})
```

Server status summary:

```
db.serverStatus().version
```

Part 12: MongoDB Database Tools

The following commands must be run from the **operating-system terminal**, not from inside `mongosh`. `mongoexport` and `mongoimport` handle JSON, CSV and TSV data interchange, while `mongodump` and `mongorestore` provide binary dump and restore capabilities.

Use this connection string:

```
mongodb://admin:admin123@localhost:27017/?authSource=admin
```

62. Export a collection to JSON

```
mongoexport
--uri="mongodb://admin:admin123@localhost:27017/?authSource=admin"
--db=smartRetailDB
--collection=products
--out=products.json
--jsonArray
```

Verify that `products.json` was created.

63. Export selected fields to CSV

```
mongoexport
--uri="mongodb://admin:admin123@localhost:27017/?authSource=admin"
--db=smartRetailDB
--collection=customers
--type=csv
```

```
--fields=customerId,name,email,address.city,premiumMember
--out=customers.csv
```

64. Import JSON data

Import the exported products into a new collection:

```
mongoimport
  --uri="mongodb://admin:admin123@localhost:27017/?authSource=admin"
  --db=smartRetailDB
  --collection=products_imported
  --file=products.json
  --jsonArray
```

Verify inside `mongosh`:

```
use smartRetailDB

db.products_imported.countDocuments()
```

65. Create a database backup

```
mongodump
  --uri="mongodb://admin:admin123@localhost:27017/?authSource=admin"
  --db=smartRetailDB
  --out=./mongodb-backup
```

The backup directory should contain BSON and metadata files.

66. Restore the database under another name

```
mongorestore
  --uri="mongodb://admin:admin123@localhost:27017/?authSource=admin"
  --nsFrom="smartRetailDB.*"
  --nsTo="smartRetailRestoreDB.*"
  ./mongodb-backup/smartRetailDB
```

Verify inside `mongosh` :

```
show dbs  
  
use smartRetailRestoreDB  
  
show collections
```

67. Replace existing data during restoration

The following command removes matching target collections before restoring them:

```
mongorestore  
  --uri="mongodb://admin:admin123@localhost:27017/?authSource=admin"  
  --drop  
  ./mongodb-backup/smartRetailDB
```

Use `--drop` carefully because it deletes the target collections before restoration.

68. Monitor MongoDB with `mongostat`

`mongostat` displays a quick operational overview of a running MongoDB instance.

```
mongostat  
  --uri="mongodb://admin:admin123@localhost:27017/?authSource=admin"  
  --rowcount=5
```

Participants should observe:

- Insert activity
- Query activity
- Update activity
- Delete activity
- Network traffic
- Memory usage
- Number of connections

69. Monitor collection activity with `mongotop`

`mongotop` reports how much time MongoDB spends reading from and writing to individual collections.

```
mongotop
--uri="mongodb://admin:admin123@localhost:27017/?authSource=admin"
5
```

The number `5` requests an updated report every five seconds.

Part 13: Final Business Requirements

Participants must implement the following requirements without copying the earlier commands directly.

Requirement 1: Product search

Create queries to display:

1. Active products costing below ₹5,000.
 2. Products belonging to Electronics or Home.
 3. Products with fewer than 30 units in stock.
 4. Products tagged as Bluetooth.
 5. The three most expensive products.
-

Requirement 2: Customer analysis

Create queries to display:

1. Premium customers.
 2. Customers from Maharashtra.
 3. Customers between 25 and 35 years of age.
 4. Customers interested in Books.
 5. Customers who have never logged in.
-

Requirement 3: Order management

Perform the following operations:

1. Change order `05006` from `PROCESSING` to `SHIPPED`.
2. Add a shipping date and courier name.

3. Change the order to `DELIVERED`.
 4. Add a delivery date.
 5. Reduce product stock based on the delivered quantity.
 6. Verify every update.
-

Requirement 4: Sales reporting

Create aggregation pipelines to determine:

1. Total number of orders.
 2. Total non-cancelled revenue.
 3. Revenue by city.
 4. Revenue by product category.
 5. Best-selling product by quantity.
 6. Customer with the highest total spending.
 7. Average value of delivered orders.
 8. Number of orders placed each month.
-

Requirement 5: Database optimisation

1. Identify three common SmartRetail queries.
 2. Create suitable indexes for those queries.
 3. Run `explain("executionStats")` before creating the indexes.
 4. Run the same analysis after creating the indexes.
 5. Compare:
 6. Execution stage
 7. Documents examined
 8. Keys examined
 9. Results returned
 10. Explain why each index was selected.
-

Requirement 6: Data management

1. Export the products collection to JSON.
2. Export customer information to CSV.
3. Create a complete database dump.
4. Restore the dump into another database.
5. Run `mongostat`.
6. Run `mongotop`.
7. Record observations from both monitoring utilities.

Part 14: Advanced Challenge

SmartRetail wants to identify customers eligible for a loyalty reward.

A customer is eligible when:

- The customer is a premium member.
- The customer has at least one non-cancelled order.
- Total spending is at least ₹5,000.

Create an aggregation pipeline that produces:

```
{
  customerId: "C101",
  customerName: "Aarav Sharma",
  totalOrders: 2,
  totalSpent: NumberDecimal("67700.00"),
  rewardStatus: "ELIGIBLE"
}
```

The pipeline should use:

- \$match
- \$group
- \$lookup
- \$unwind
- \$project
- \$cond
- \$sort

Part 15: Expected submission

Participants should submit:

1. MongoDB Shell command history.
2. Screenshots of the created collections.
3. CRUD operation results.
4. At least ten query examples.
5. At least five update examples.
6. Safe-delete demonstration.
7. Five aggregation reports.
8. Index definitions.

9. Before-and-after query execution plans.
 10. Exported JSON and CSV files.
 11. Database backup directory.
 12. Monitoring observations.
 13. Final loyalty-reward aggregation pipeline.
-

Part 16: Assessment rubric

Area	Marks
Shell and database setup	10
BSON data types and document design	10
Create and read operations	10
MongoDB queries	15
Update operations	10
Delete operations	5
Aggregation framework	20
Indexes and query analysis	10
Utilities and backup	5
Advanced challenge	5
Total	100

Part 17: Key business outcome

After completing the case study, SmartRetail will have:

- A flexible product catalogue
- Structured customer profiles
- Embedded order-item information
- Order and payment tracking
- Inventory-update capabilities
- Sales-analysis pipelines
- Optimised search operations
- Data export and import capability
- Backup and restore procedures
- Basic database-monitoring procedures